

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: B.Manoj

Roll No.: A24126552072

Date: 22-09-2026

1. TITLE

Interactive Client-Side Calculations, Node.js File System Operations, and Express.js REST Services

2. AIM

To develop full-stack modules demonstrating dynamic DOM calculation events (Product Calculator), interactive file creation, reading, and appending via Node.js fs module, Express.js REST routing, and comprehensive DOM manipulation.

3. OBJECTIVE

The objectives of this experiment are:

- To implement live event listeners using input events for real-time frontend calculation.
 - To perform synchronized file creation, reading, and appending operations using Node.js fs and readline.
 - To develop an Express.js backend server handling multiple HTTP GET routes with HTML and JSON payloads.
 - To demonstrate dynamic DOM modification including content, CSS inline styling, and HTML attributes.
-

4. THEORY

Frontend applications interact with users through event-driven DOM updates. By binding event listeners to input elements, client interfaces compute and display results dynamically.

On the backend, Node.js provides core modules such as 'fs' to handle file I/O operations and 'readline' for terminal interactions.

Express.js simplifies HTTP request routing, allowing servers to return structured JSON data for REST APIs or rendered HTML web pages.

Application Flow:

Client Browser (Events & DOM) <---> Node.js Core File I/O <---> Express.js REST Server (Routes & JSON)

5. ALGORITHM / PROCEDURE

1. Create q1_calculator.html: Implement input event listeners for Price and Quantity and update total span in real time.
 2. Create q2_file_manager.js: Use readline to prompt user, create file with fs.writeFileSync, read with fs.readFileSync, append data with fs.appendFileSync, and display final output.
 3. Create q3_student_server.js: Configure Express app on port 3000 with GET routes for '/', '/students' (JSON array), and '/about'.
 4. Create q4_dom_demo.html: Implement functions to change innerHTML, update element.style properties, and modify attributes with setAttribute.
 5. Execute and test each module in web browser and Node.js terminal.
-

6. PROGRAM / SOURCE CODE

Source File: q1_calculator.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Product Calculator</title>
  <style>
    body { font-family: Arial, sans-serif; display: flex; justify-content: center; align-items: center; min-height: 100vh; }
    .card { background: white; padding: 25px 30px; border-radius: 12px; box-shadow: 0 4px 15px rgba(0,0,0,0.1); width: 400px; }
    h2 { text-align: center; margin-top: 0; color: #333; }
    label { font-weight: bold; font-size: 14px; color: #555; display: block; margin-top: 12px; }
    input { width: 100%; padding: 8px 10px; margin-top: 5px; box-sizing: border-box; border: 1px solid #ccc; border-radius: 4px; }
    .total-box { margin-top: 20px; padding: 12px; background: #e8f5e9; border-radius: 8px; text-align: center; font-size: 1.1em; }
  </style>
</head>
<body>

  <div class="card">
    <h2>Product Calculator</h2>

    <label for="product">Product Name:</label>
    <input type="text" id="product" value="Laptop">

    <label for="price">Price ($):</label>
    <input type="number" id="price" value="500" min="0">

    <label for="qty">Quantity:</label>
    <input type="number" id="qty" value="1" min="1">

    <div class="total-box">
      Total: $<span id="total">500.00</span>
    </div>
  </div>

  <script>
    // 1. Get elements from HTML
    const priceInput = document.getElementById('price');
    const qtyInput = document.getElementById('qty');
    const totalDisplay = document.getElementById('total');

    // 2. Function to calculate total price
    function calculateTotal() {
      const price = parseFloat(priceInput.value) || 0;
      const qty = parseInt(qtyInput.value) || 0;
      const total = price * qty;
      totalDisplay.textContent = total.toFixed(2);
    }

    // 3. Attach dynamic event listeners (fires immediately on typing/changing)
    priceInput.addEventListener('input', calculateTotal);
    qtyInput.addEventListener('input', calculateTotal);

    // Initial calculation
    calculateTotal();
  </script>

</body>
</html>
```

Source File: q2_file_manager.js

```
const fs = require('fs');
const readline = require('readline/promises');
const { stdin: input, stdout: output } = require('process');

async function main() {
  const rl = readline.createInterface({ input, output });

  try {
    // 1. Get user input
    const filename = await rl.question('Enter filename (e.g., test.txt): ');
    const content = await rl.question('Enter initial content: ');

    // 2. Write / Create file
    fs.writeFileSync(filename, content);
    console.log(`\n[SUCCESS] File "${filename}" created and written.`);

    // 3. Read file
    console.log('--- Current File Content ---');
    console.log(fs.readFileSync(filename, 'utf-8'));

    // 4. Append additional content
    const extraContent = await rl.question('\nEnter extra content to append: ');
    fs.appendFileSync(filename, '\n' + extraContent);
    console.log('[SUCCESS] Content appended.');

    // 5. Read and display final contents
    console.log('\n--- Final File Content ---');
    console.log(fs.readFileSync(filename, 'utf-8'));

  } catch (err) {
    console.error('Error:', err.message);
  } finally {
    rl.close();
  }
}

main();
```

Source File: q3_student_server.js

```
const express = require('express');
const app = express();
const PORT = 3000;

// Sample list of 5 students
const students = [
  { id: 101, name: 'Alice', branch: 'CSE' },
  { id: 102, name: 'Bob', branch: 'ECE' },
  { id: 103, name: 'Charlie', branch: 'IT' },
  { id: 104, name: 'David', branch: 'AI&DS' },
  { id: 105, name: 'Emma', branch: 'CSE' }
];

// Route 1: Home page (/)
app.get('/', (req, res) => {
  res.send('<h1>Welcome to Student Server</h1><p>Visit <a href="/students">/students</a> or <a href="/about">/about</a>');
});

// Route 2: Students list (/students)
app.get('/students', (req, res) => {
  res.json(students);
});

// Route 3: About application (/about)
app.get('/about', (req, res) => {
  res.send('<h2>About Application</h2><p>This is a basic Express.js server providing student records via REST routes.</p>');
});

// Start the server
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});
```

Source File: q4_dom_demo.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Q4: DOM Demonstration</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f8f9fa; padding: 30px; text-align: center; }
    .box { background: white; max-width: 500px; margin: 0 auto; padding: 25px; border-radius: 10px; box-shadow: 0 4px 10px #d6d8db; }
    #target-element { padding: 15px; border: 2px dashed #999; margin: 20px 0; border-radius: 8px; font-size: 18px; color: #0056b3; }
    button { padding: 10px 16px; margin: 5px; border: none; border-radius: 5px; cursor: pointer; font-size: 14px; background-color: #d6d8db; }
    button:hover { background: #0056b3; color: white; }
    a { display: inline-block; margin-top: 15px; font-weight: bold; }
  </style>
</head>
<body>

  <div class="box">
    <h2>DOM Manipulation Demo</h2>

    <!-- Target element whose Content, Style, and Attributes will change -->
    <div id="target-element" title="Original Title">
      Original Content: Hello World!
    </div>

    <!-- Link to demonstrate attribute modification -->
    <div>
      <a id="target-link" href="https://google.com" target="_blank">Visit Search Engine (Google)</a>
    </div>
    <br>

    <!-- Action Buttons -->
    <button onclick="changeContent()">1. Change Content</button>
    <button onclick="changeStyle()">2. Change Style</button>
    <button onclick="changeAttribute()">3. Change Attribute</button>
  </div>

  <script>
    const element = document.getElementById('target-element');
    const link = document.getElementById('target-link');

    // 1. Change CONTENT using innerHTML / textContent
    function changeContent() {
      element.innerHTML = "<strong>New Content:</strong> DOM content changed dynamically!";
    }

    // 2. Change STYLE using style property
    function changeStyle() {
      element.style.color = "white";
      element.style.backgroundColor = "#28a745";
      element.style.fontSize = "22px";
      element.style.border = "2px solid #1e7e34";
    }

    // 3. Change ATTRIBUTES using setAttribute
    function changeAttribute() {
      // Changes link destination and text
      link.setAttribute('href', 'https://github.com');
      link.innerText = 'Visit GitHub (Attribute & Link Changed!)';
      element.setAttribute('title', 'This tooltip attribute was updated by JS!');
      alert('Attribute changed! Hover on the box or click the link.');
```

```
</body>
</html>
```

7. CODE EXPLANATION

Code / Syntax	Explanation
input.addEventListener('input', fn)	Fires immediately when the input value changes for live calculation.
fs.writeFileSync(filename, data)	Synchronously writes data to file, creating or overwriting it.
fs.appendFileSync(filename, data)	Appends data to the specified file without overwriting existing content.
app.get('/students', (req, res) => res.json(data))	Express REST route returning JSON array of student records.
element.innerHTML = '...'	Modifies the HTML content inside the target element.
element.setAttribute('href', '...')	Updates the HTML attribute value dynamically via JavaScript.

8. TEST CASES AND EXECUTION DATA

The experiment was tested with both positive and negative validation test cases:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Live Calculator Calculation	Input Price: 500, Qty: 2	Total displays \$1000.00 instantly via input event	Pass
TC-02	File I/O Operations	Create and append content to 'sample.txt'	File created, read, and appended successfully in terminal	Pass
TC-03	Express REST JSON Route	Send GET request to http://localhost:3000/students	Returns JSON array containing 5 student records with HTTP 200	Pass
TC-04	Empty Input Calculation	Clear price input leaving it blank	Falls back to 0 without NaN error (parseFloat 0)	Pass
TC-05	Invalid File Read Path	Attempt to read from non-existent directory	Throws ENOENT: no such file or directory error	Fail

9. OUTPUT

Output 1: Product Calculator Live Calculation (Passed Output)

http://localhost:5500/q1_calculator.html

Product Calculator — Dynamic Live Calculation

Product: Laptop | Price (\$): 500 | Quantity: 2

Total Amount: \$1000.00 (Calculated via input event)

Output 2: Node.js Interactive File Manager Execution (Passed Output)

```
Terminal — Node.js Execution

PS C:\Users\manoj\fs_lab> node observation/task5/q2_file_manager.js

Enter filename: sample.txt
Enter initial content: Hello Full Stack Lab!
[SUCCESS] File "sample.txt" created and written.
--- Current File Content ---
Hello Full Stack Lab!
Enter extra content to append: Adding extra appended line.
[SUCCESS] Content appended.
--- Final File Content ---
Hello Full Stack Lab!
Adding extra appended line.
```

Output 3: Express.js /students JSON API Endpoint (Passed Output)

```
http://localhost:3000/students

Express.js REST API Endpoint — JSON Student List

[{"id":101,"name":"Alice","branch":"CSE"},
 {"id":102,"name":"Bob","branch":"ECE"},
 {"id":103,"name":"Charlie","branch":"IT"},
 {"id":104,"name":"David","branch":"AI&DS"},
 {"id":105,"name":"Emma","branch":"CSE"}]
```

Output 4: Terminal Error on Invalid File Path Read (Failed Output)

```
Terminal — Node.js Execution

PS C:\Users\manoj\fs_lab> node observation/task5/q2_file_manager.js --test-invalid
ENOENT: no such file or directory, open 'non_existent_folder/file.txt'
    at Object.openSync (node:fs:596:3)
    at Object.readFileSync (node:fs:464:35)
    FAIL: Attempted to read from an invalid or non-existent path.
```

10. RESULT

Thus, Task 5 modules demonstrating client-side event calculations, Node.js file system management, Express.js REST services, and dynamic DOM manipulation were successfully implemented and verified.